

World Cup 2026 Project

A line-by-line walkthrough, explained from scratch

This guide explains, in plain language, the football data project you built: what the ideas mean, what every tool does, what your data is, and what your code does one piece at a time. No prior data-science knowledge assumed. Read it top to bottom and you will understand your own project well enough to explain it to anyone.

Built with Python, pandas, NumPy and matplotlib. @Kishloy_

1. What you built, in one breath

You took football numbers and turned them into charts and a prediction. The project looks at the 2026 World Cup through **two different lenses**, then compares them.

- **Lens 1, what teams actually did.** Using real qualifying match data, you measured how good each team has been, adjusted for how tough their opponents were. Think of it as a school report card based on past games.
- **Lens 2, what is projected to happen.** Using the output of a tournament simulation, you showed each team's title odds, strength rating and chances of escaping their group. Think of it as a bookmaker's forecast.
- **The bridge.** You put the two side by side to see who the forecast trusts more than their actual numbers, and who is being helped by an easy draw.

Why two lenses? A report card tells you what already happened. A forecast guesses the future. They are different questions, and the interesting bit is where they disagree.

2. The ideas you need first

Expected goals (xG)

xG is a number for **how good your chances were**, not how many you actually scored. Every shot is given a value between 0 and 1 for how likely it was to be a goal. Add them up for a match and you get xG. A tap-in might be 0.8, a hopeful shot from distance 0.03. If your shots add up to 1.5, your xG is 1.5, even if you scored 0 or 3. It measures the **quality of chances**, which is steadier and more telling than the scoreline.

xGA (expected goals against)

The same idea, but for the chances you let the other team have. **Lower is better**, it means you gave up fewer good chances.

Why the opponent matters (the problem you solved)

Putting up 2.0 xG against a weak team is not the same as 2.0 xG against Spain. If you only average the raw numbers, a team that bullied minnows looks elite and a team that battled giants looks poor. Your project corrects for this, so the numbers are fair across different schedules. This

adjustment is the original, your-own-work part.

FIFA ranking

A list that ranks national teams by strength. **A lower number means a stronger team** (rank 1 is the best in the world). You used each opponent's rank as a stand-in for how hard that match was.

Ratings: Supremacy and Index

Single numbers from the simulation that sum up a team's strength. Higher means stronger.

Supremacy is a net rating (roughly attack minus defence), so a positive number is an above-average team and a negative number is below average.

Monte Carlo simulation

A way to deal with uncertainty: get a computer to play the whole tournament many thousands of times, with a little randomness each time, then count how often each team wins. If Spain win 14 times out of every 100 runs, their win probability is 14%. It is the same logic as rolling dice over and over to learn the odds.

Probability

A number from 0 to 1 (or 0% to 100%) for how likely something is. A group's four finishing chances (1st, 2nd, 3rd, 4th) always add up to 1, because the team has to finish somewhere.

3. The tools

- **Python** is the programming language everything is written in.
- **Jupyter Notebook** is where you write code in small chunks ("cells"), run them, and see charts appear right underneath. Like a lab notebook that also does the maths.
- **pandas** gives you a DataFrame, which is just a table (rows and columns) you can sort, filter and do sums on, in code.
- **NumPy** does fast maths on whole lists of numbers at once, like averaging a thousand values in one line.
- **matplotlib** is the drawing tool that turns numbers into the charts.

4. The data you used, and how it is stored

Most of your data lives in **dictionaries**. A dictionary is a set of labelled boxes written as key: value. You look things up by name. Here is the shape of your match data, RAW_MATCHES:

```
RAW_MATCHES = {
    'Spain': [(1.37, 0.95, 11), (1.37, 1.28, 55), (4.79, 0.26, 69), ...],
    'Brazil': [(3.72, 0.20, 62), (0.91, 0.16, 64), ...],
    ...
}
```

The key is the team name. The value is a list of matches. Each match is a tuple of three numbers in a fixed order: **(xGF, xGA, opponent rank)**. So Spain's first entry means: in that game Spain created 1.37 xG, allowed 0.95 xGA, against the team ranked 11th. You typed in 1025 of these across 46 teams.

- RAW_MATCHES holds the match-by-match xG, the raw material for Lens 1.
- BASE holds each team's group letter plus extras like shots and corners.
- SIM holds the simulation output for all 48 teams: title odds, ratings, group goals and finishing probabilities. This is the external data, the part you credit.

Honesty note: The match xG and the adjustment are your work. The simulation numbers came from an external source, so they are credited and kept clearly separate in the project.

5. Walking through the code

5.1 Setup

Every notebook starts by loading its tools and setting a house style.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

These three lines bring in your tools and give them short nicknames, so later you can type `pd` instead of `pandas`. This is standard convention.

```
GROUP_COLORS = {'A': '#e74c3c', 'B': '#e67e22', ... }
BG, PANEL = '#0d1117', '#161b22'
CREDIT_XG = 'Opponent-adjusted xG from qualifying | @Kishloy_'
```

These are just stored settings: a colour for each group, the dark background colours, and the credit text that sits at the bottom of every chart. Storing them once means every chart looks consistent.

5.2 The heart: the opponent adjustment

This is the most important function, `build_xg()`. Its job: turn raw match xG into a fair, opponent-adjusted number for each team. Let us take it in four steps.

Step 1, pool every match into three lists.

```
A=[]; D=[]; L=[]
for ms in RAW_MATCHES.values():
    for a,d,r in ms:
        A.append(a)          # all xGF values
        D.append(d)          # all xGA values
        L.append(np.log(r))  # log of every opponent rank
```

It walks through every team, then every match, and collects three big lists: all the xGF numbers (A), all the xGA numbers (D), and the opponent ranks (L). The `np.log(r)` bit takes the **logarithm** of the rank. In plain terms, log squashes big numbers so the gap between rank 1 and rank 20 counts for more than the gap between rank 150 and 170, which matches how football actually works.

Step 2, find the league averages and the league trend.

```
g_xgf, g_xga = A.mean(), D.mean()
ba, ia = np.polyfit(L, A, 1)
bd, idd = np.polyfit(L, D, 1)
```

`A.mean()` is just the average xGF across all 1025 matches, the "typical" attacking output. `np.polyfit(L, A, 1)` draws the **line of best fit** through the cloud of (opponent rank, xGF) dots. It

answers: "for an average team, how much xG would you expect against an opponent of this rank?" It returns two numbers, a slope (ba) and a starting point (ia), which together describe that line. The second pair (bd, idd) does the same for defence.

Step 3, score each team against that expectation.

```
a = np.array([m[0] for m in ms]) # this team's xGF
r = np.array([m[2] for m in ms]) # this team's opponents
Lt = np.log(r)
adj_xg = g_xgf + (a - (ba*Lt + ia)).mean()
```

For one team, $ba \cdot Lt + ia$ is the line's **prediction**: what an average team would have managed against those same opponents. $a - \text{prediction}$ is the **residual**, how much the team beat or missed that expectation in each game. Average those residuals, add them to the league average g_xgf , and you get their xG "re-based onto a neutral schedule." Defence works the same way.

In one sentence: The adjusted number says how a team would score against an average opponent, by measuring how far above or below the league trend they performed.

Step 4, handle teams with no data and return a table.

```
else: # team has no match sheet
    rows.append(dict(team=t, adj_xg=b['old_xg'], adjusted=False, ...))
return pd.DataFrame(rows)
```

Two teams (Scotland and Ivory Coast) had no match data, so they keep their old values and get a flag `adjusted=False` so you can mark them on charts. Finally `pd.DataFrame(rows)` turns the pile of rows into a neat table you can sort and plot.

5.3 The simulation table

This function reads the simulation dictionary and works out a few handy columns.

```
win_prob = 1.0 / d['win_odds']
sim_xg    = d['g_gf'] / 3
reach_ko  = p1 + p2
```

$1.0 / \text{win_odds}$ turns betting-style odds into a probability (odds of 7.13 means about a 1-in-7 chance, so 14%). $g_gf / 3$ turns the expected goals across three group games into a per-match number. $p1 + p2$ adds the chance of finishing first or second, the automatic route into the knockouts.

```
assert (abs(sim_df[['p1', 'p2', 'p3', 'p4']].sum(axis=1) - 1.0) <= 0.02).all()
```

An assert is a safety check: "this had better be true, or stop and warn me." Here it checks that every team's four finishing chances add up to 1. If a typo broke that, the code crashes loudly instead of quietly producing a wrong chart. Checks like this are how you "make no mistakes."

5.4 Joining the two lenses

```
m = adj.merge(sim_df, on='team')
m['obs_net'] = m['adj_xg'] - m['adj_xga'] # observed net strength
m['draw_shift'] = m['sim_xg'] - m['adj_xg'] # projection minus reality
```

`merge` glues the two tables together by team name, like a spreadsheet VLOOKUP. `obs_net` is your observed net strength from qualifying. `draw_shift` is the key bridge number: how much the simulation's projected attack differs from what the team actually did. A big positive means an easy

draw is expected to flatter them; a big negative means their qualifying numbers were padded against weak teams.

5.5 Making a chart (one example, the rest follow the same recipe)

```
fav = sim_df.nlargest(15, 'win_prob').sort_values('win_prob')
fig, ax = plt.subplots(figsize=(12, 9))
bars = ax.barh(fav['team'], fav['win_prob']*100)
for bar, val in zip(bars, fav['win_prob']*100):
    ax.text(val+0.15, bar.get_y(), f'{val:.1f}%')
ax.set_title('WC2026: Title Favorites')
plt.savefig('m4_favorites.png', dpi=200)
```

Line by line: `nlargest(15, ...)` takes the 15 most likely winners. `plt.subplots()` makes a blank canvas. `ax.barh(...)` draws horizontal bars, team names down the side, win % along the bottom. The for loop writes the percentage at the end of each bar. `set_title` adds the heading, and `savefig` saves it as an image at 200 dots-per-inch (sharp enough to post). Every other chart in the project is this same pattern with different data and chart type.

5.6 The bracket prediction

```
def win(a, b):
    return a if sup[a] >= sup[b] else b
```

This tiny function is the whole prediction engine: given two teams, return whichever has the higher Supremacy rating. To run a round you apply it to every pairing; to run the tournament you repeat that for the Round of 16, quarters, semis and final. Because it always picks the stronger team, the bracket has no upsets, which is why it is a "most likely" path rather than a guaranteed one.

6. The adjustment, with tiny made-up numbers

To really see it, here are two pretend teams. Say the league average xG is **1.66**, and the trend says an average team would manage about **1.1** xG against a top opponent and **2.1** against a weak one.

- **Team A** recorded 1.2 xG against a strong team and 2.0 against a weak team. Raw average = 1.6.
- **Team B** recorded 1.2 xG and 2.0 xG too, raw average also 1.6, but **both** games were against weak teams.

Team A: against the strong team they got 1.2 versus an expected 1.1, so +0.1. Against the weak team they got 2.0 versus an expected 2.1, so -0.1. Average residual = 0. Adjusted xG = 1.66 + 0 = **1.66**, bang average.

Team B: both games expected about 2.1, they got 1.2 and 2.0, residuals -0.9 and -0.1, average -0.5. Adjusted xG = 1.66 - 0.5 = **1.16**, below average.

The point: Same raw average (1.6) for both, but once you account for who they played, Team A is average and Team B is below it. That is exactly the unfairness the adjustment removes.

7. What each chart is telling you

- **Attack vs Defense map:** every team plotted by adjusted attack and defence. Top right is elite.
- **Top attacks / defenses:** the best ten at each end, after the opponent adjustment.
- **Title favourites:** the simulation's win probabilities.
- **Power ranking:** all 48 teams by Supremacy rating.
- **Group of death:** groups ranked by combined team strength.
- **Qualification:** each group's finishing-position probabilities.
- **Confederation strength:** average team strength by continent.
- **Bridge scatter & draw effect:** where the forecast and the real numbers agree or clash.

8. Glossary

xG	expected goals: the quality of chances created.
xGA	expected goals against: the quality of chances allowed (lower is better).
Opponent adjustment	re-scaling a team's numbers to be fair across easy and hard schedules.
Residual	how much a team beat or missed the average expectation in a game.
Supremacy	a net team-strength rating from the simulation (higher is stronger).
Monte Carlo	playing the tournament many times on a computer to estimate odds.
DataFrame	a table of data in pandas.
Dictionary	labelled boxes of data written as key: value.
assert	a safety check that stops the code if something is not true.

9. Where to take this next

- Learn the pandas basics properly (selecting columns, sorting, grouping), since that is most of real data work.
- Try changing one chart, for example colour the favourites by confederation instead of group, to get comfortable editing matplotlib.
- Read a gentle intro to linear regression, which is exactly the line-of-best-fit your adjustment uses.
- As a stretch, try writing a tiny Monte Carlo of your own: simulate one group 10,000 times and count finishes.

You built the data pipeline, the opponent-adjusted xG analysis, and every visualization in this project. That is real, defensible work. Be proud of it, and credit the simulation data as external.
@Kishloy_